Analysis and Design of Algorithms

CS3230

Recitation

Week 7

Prof Lee Wee Sun

Select

Find the element of rank k.

Randomized select (quickselect) runs in $O(n)$ expected time.

Worst case linear time select runs in $O(n)$ in the worst case (but practically slow).

These algorithms are example of divide and conquer and randomized algorithms.

Algorithms commonly used as subroutines for solving problems

- Binary Search
- Sorting
- Select

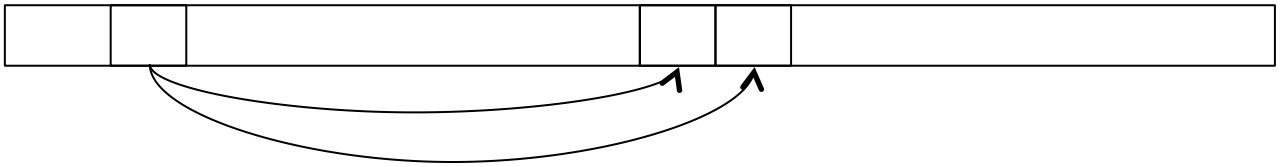
Often overlooked

Programming Assignment 1

- **Task A**: Implementing Strassen's algorithm.
- Learning points
  - In practice, can often obtain speedup by switching to a different algorithm when the problem is small enough.
    - Switch to algorithms that may be asymptotically slower but are fast for small problems.
    - E.g. switch to insertion sort when running quicksort or merge sort.
  - System issues matter, e.g. cache.
    - Simpler algorithms often have better memory access patterns and can be competitive until fairly large problem sizes.

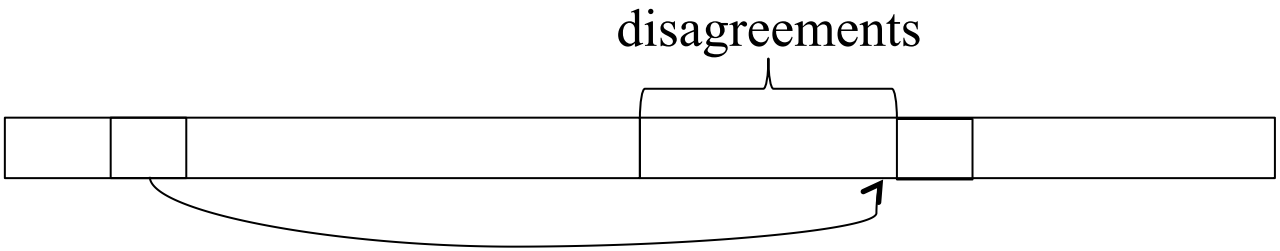
- **Task B**. Count disagreements between rankings.
- Sequential solution: go through each pair and check whether disagree in $\Theta(n^2)$ time.
- Divide and conquer solution
  - Divide array in the middle.
  - Recursively count num of disagreements in left subproblem and right subproblems
  - Add count of disagreements across subproblems.

- Naïve way of computing disagreements across subproblems: $\Theta(n^2)$



  - Runtime: $T(n) = 2T(n/2) + \Theta(n^2) = \Theta(n^2)$
  - No gain compared to sequentially computing disagreements.

- Better way:
  - Keep subproblem sorted after counting disagreements using merge in $\Theta(n)$ time.
  - For each element in left subarray, use binary search to find potential insertion location in right subarray in $\Theta(\lg n)$ time.



  - Runtime: $T(n) = 2T(n/2) + \Theta(n \lg n) = \Theta(n \lg^2 n)$

- Even better way:
  - Keep subproblem sorted after counting disagreements using merge in $\Theta(n)$ time.
  - For each element in left subarray, compute potential insertion location in right subarray as part of the merge process.
  - Runtime: $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \lg n)$

Admin

- Reminder: Midterm test
  - Saturday 1 Oct 10am to 11.30am.
  - Please arrive 10 minutes before 10am
  - Closed book, with 1 handwritten (on both sides) crib sheet allowed
  - Calculator allowed
- Please do the Midterm Revision Exercise on Coursemology before the midterm (Trainings -> Test tab)
- Homework 2 due on Friday midnight.

MCQ

Question 1

Both randomized select and randomized quicksort are divide and conquer algorithms. Which of the following is true?

- Randomized select recurses on smaller subproblems.
- Randomized select recurses on fewer subproblems.
- Randomized select's runtime for the divide and combine step is asymptotically smaller.

Solution

Both the algorithms randomly select a pivot to partition the array.

Randomized quicksort recurses on both partitions resulting in an expected runtime of $\Theta(n \lg n)$.

Randomized quickselect recurses on only one of the subproblems resulting in an expected runtime of $\Theta(n)$.

Question 2

An adversary can construct an input of size n to force randomized quickselect to run in $\Omega(n^2)$ time.

- True
- False

Solution

False. As the adversary does not know the randomization that is being used, the adversary cannot construct an input that will run in $\Omega(n^2)$ time.

Note that it is possible that some runs of the algorithm will take $\Omega(n^2)$ time due to the randomization done by the algorithm, but the adversary has no way to force this for any run of the algorithm.

Question 3

The tertiles of a set of values are the two values that divides the set into three parts, with the first tertile separating the smallest one-third of the values from the other values and the second tertile separating the largest one-third of values from the other values. Computing the tertiles of a set of n values takes $\Omega(n \lg n)$ time for any algorithm working in the comparison-based model.

- True
- False

Solution

1. Use select to find the $1/3n$-th order statistic in $\Theta(n)$ time
2. Run partition to extract the first $1/3n$ elements in $\Theta(n)$ time.
3. Run select to find the median of the remaining $2/3n$ of the elements in $\Theta(n)$ time.

Total runtime is $\Theta(n)$.

Question 4

Consider comparison-based algorithms for finding the first order statistic. What is the height of the decision tree describing the optimal algorithm?

- $\Theta(\lg n)$
- $\Theta(n)$
- $\Theta(n^2)$

Solution

- The 1st order statistic is the smallest element.
- We can find the smallest element by keeping the smallest found so far while going through the array. This takes n-1 comparisons; the tree has height at most n-1.
- We also require at least n-1 comparisons. Otherwise there is an element that is not compared, and that element can be changed to be made smaller than the smallest element.

Question 5

Who is the Master of Algorithms pictured below?

- Robert Tarjan
- Robert Floyd
- Richard Karp
- Tony Hoare

Robert Floyd

Turing Award winner

- One of the inventors of linear time select (Blum, Floyd, Pratt, Rivest, Tarjan)
- Known for Floyd-Warshall algorithm for all-pairs shortest path, Floyd-Hoare logic (use invariants for correctness)
- Did not have a PhD!

Structured Questions

Question 1(a)

(CLRS 9.3-5) Suppose you have a ``black box" worst case linear time median subroutine. Give a simple linear time algorithm that solves the selection problem for arbitrary order statistics.

Solution:

```
SELECT(A, low, high, i)
    if low == high
        return A[low]
    pivot = MEDIAN(A, low, high)
    PARTITIONWITHPIVOT(A, low, high, pivot)
    k = pivot − low + 1
    if i == k
        return A[pivot]
    elseif i < k
        return SELECT(A, low, pivot − 1, i)
    else return SELECT(A, pivot + 1, high, i − k)
```

$$T(n) = T(n/2) + \Theta(n)$$

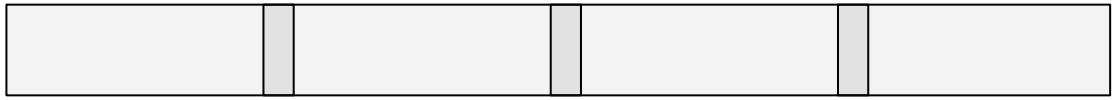
$$f(n) = n = \Omega(n^{\log_2 1+\epsilon})$$

Reg. Cond: $f(n/2) = n/2 = f(n)/2$

Master Thm Case 3: $T(n) = \Theta(n)$

Question 1(b)

(CLRS 9.3-6) The kth-quantiles of an n element set are the k-1 order statistics that divide the sorted set into k equal sized sets (to within 1). Give a $O(n \lg k)$-time algorithm to list the kth-quantiles of a set.



4-th quantiles

Solution:

First find the k-1 orders corresponding the the kth-quantiles.

- Let $r = n \% k$
- Set the first r order statistics as the i*($floor$(n/k) + 1)-th order statistics for i = 1, ... , r, and the remaining k-r-1 orders at increment of $floor$(n/k).
- Put the orders in sequence in an array B and the original numbers in an array A.

Do a divide and conquer algorithm *FindQuantiles*(A,B).

1. First find the middle element $B[mid]$.
2. Find the $B[mid]$-th order statistics using the select algorithm in linear time.
3. Use the element as pivot to partition A in linear time into
   - A' :elements smaller than the pivot element
   - A'': elements larger than the pivot element.
4. Divide the elements in B into
   - B' : elements smaller than $B[mid]$
   - B'' : elements $B[j]$-$B[mid]$ for $j > mid$.
5. Recurse on *FindQuantiles*(A',B') and *FindQuantiles*(A'', B'')
6. Return union of the order statistics

Runtime:

- Height of the recursion tree is $O(\lg k)$ as we are repeated dividing the set of elements into 2.
- Each level of the recursion tree has total computation time of $O(n)$.
- Total computation time is $O(n \lg k)$.

Why not sequentially find the order statistics instead of doing divide and conquer?

Question 1(c)

(CLRS 9.3.7) Describe an $O(n)$-time algorithm that, given a set S of n distinct numbers and a positive integer $k \leq n$, determines the k numbers in S that are closest to the median of S.

1. Find the median in $O(n)$ time.
2. Go through the array and compute the distance of each element to the median in $O(n)$ time and store it in a distance array D with pointers to the original array.
3. Find the $(k+1)$-st order statistic in D in $O(n)$ time
4. Run the partition algorithm in $O(n)$ time to get the $(k+1)$ elements (including the median) that are closest to the median.

Each step takes $O(n)$ time giving a $O(n)$-time algorithm

Question 1(d)

Given a set of points $p_1, \ldots, p_n$ on the real line, give a fast algorithm to find a point x that minimizes

$$\sum_{i=1}^{n} |p_i - x|$$

(For convenience, you may assume that n is odd.)

Solution: Setting x to the median minimizes

$$\sum_{i=1}^{n} |p_i - x|$$

To see this, consider

$$\sum_{i=1}^{n} |p_i - y| - \sum_{i=1}^{n} |p_i - x|$$

for some $y \neq x$. We argue that this is greater than or equal to zero.

x (median) $\qquad\qquad$ $y \geq x$

$\lvert p_i - y\rvert - \lvert p_i - x\rvert$ $= y - x$ for $p_i < x$	$\lvert p_i - y\rvert - \lvert p_i - x\rvert$ $\leq y - x$ for $x < p_i < y$	$\lvert p_i - y\rvert - \lvert p_i - x\rvert$ $= x - y$ for $p_i > y$

Consider the case where $y \geq x$.

$$\sum_{i=1}^{n} |p_i - y| - \sum_{i=1}^{n} |p_i - x|$$

$$\geq \sum_{i=1}^{\lfloor n/2 \rfloor} (y - x) - \sum_{\lceil n/2 \rceil + 1}^{n} (y - x)$$

$$= 0$$

The case $y \leq x$ is similar. Run select to get the median in linear time.